

PRACTICAL 18 -

Objective - The name roll number and marks of 10 students are stored in an array named marks. Write a program to find the Name of first, second and third rank holder.

CODE -

```
1  #include <stdio.h>
2  #include <stdbool.h>
3  #include <stdlib.h> // For malloc and free
4
5  // Function to check if a 2D matrix is sparse.
6  // Takes the matrix, number of rows, and number of columns as input.
7  bool isSparseMatrix(int **matrix, int rows, int cols) {
8      int size = rows * cols; // Total number of elements in the matrix.
9      int zeroCount = 0;      // Count of zero elements.
10
11     // Iterate through the matrix and count zero elements.
12     for (int i = 0; i < rows; i++) {
13         for (int j = 0; j < cols; j++) {
14             if (matrix[i][j] == 0) {
15                 zeroCount++;
16             }
17         }
18     }
19
20     // A matrix is sparse if the number of zero elements is greater
21     // than half of the total elements.
22     if (zeroCount > (size / 2)) {
23         return true; // Matrix is sparse.
24     } else {
25         return false; // Matrix is not sparse.
26     }
27 }
28
29 int main() {
30     int rows = 4;
31     int cols = 5;
32
33     // Dynamically allocate the matrix.
34     int **matrix = (int **)malloc(rows * sizeof(int *));
35     if (matrix == NULL) {
36         printf("Memory allocation failed.\n");
37         return 1; // Exit with an error code.
38     }
39
40     for (int i = 0; i < rows; i++) {
41         matrix[i] = (int *)malloc(cols * sizeof(int));
```

```
41     if (matrix[i] == NULL) {
42         printf("Memory allocation failed.\n");
43         // Free any already allocated memory
44         for(int k = 0; k < i ; k++){
45             free(matrix[k]);
46         }
47         free(matrix);
48         return 1;
49     }
50 }
51
52 // Initialize the matrix with sample data.
53 int sampleMatrix[4][5] = {
54     {0, 0, 3, 0, 4},
55     {0, 0, 5, 7, 0},
56     {0, 0, 0, 0, 0},
57     {0, 2, 6, 0, 0}
58 };
59
60 // Copy sample data into the dynamically allocated matrix.
61 for (int i = 0; i < rows; i++) {
62     for (int j = 0; j < cols; j++) {
63         matrix[i][j] = sampleMatrix[i][j];
64     }
65 }
66
67 // Check if the matrix is sparse and print the result.
68 if (isSparseMatrix(matrix, rows, cols)) {
69     printf("The matrix is sparse.\n");
70 } else {
71     printf("The matrix is not sparse.\n");
72 }
73
74 // Free dynamically allocated memory.
75 for (int i = 0; i < rows; i++) {
76     free(matrix[i]);
77 }
78 free(matrix);
79
80 return 0;
81 }
```

OUTPUT -

```
Enter the number of rows: 4
Enter the number of columns: 5
Enter the matrix elements:
0 0 3 0 4
0 0 5 7 0
0 0 0 0 0
0 2 6 0 0
The matrix is sparse.
```